

CSE 102 (Fall 2020) – Lab 1

Before you begin the assignment, please read the following carefully.

- Read the Homework Guidelines.
- Every part of each question begins on a new page. Do not change this.
- This does not mean that you should write a full page for every question. Your answers should be short and precise. Lengthy and wordy answers will lose points.
- Do not change the format of this document. Simply type your answers as directed.
- You are **not** allowed to work in teams.
- You **are** allowed to discuss homework problems at a high level with groups of up to 4 classmates, however any submitted work must be your own. You may not share your written solutions with one another.
- Indicate any collaborators on the line below, and in each solution specify any key ideas obtained from group discussion.
- The lab is due for submission on Gradescope at the following times
 - Section A and B: 19 October (10 a.m.)
 - Section C and D: 20 October (10 a.m.)

I have read and agree to the collaboration policy. – Baladithya Balamurugan, bbalamur@ucsc.edu
Collaborators:

1. We all know how to search a sorted 1-dimensional array. This problem involves searching two dimensional arrays that are partially sorted.

Call a two-dimensional square ($n \times n$) array A line-sorted if

- the entries of A are all distinct numbers,
- the entries in each row are sorted in ascending order (left to right), and
- the entries in each column are sorted in ascending order (top to bottom).

If A is line-sorted then both $A[i-1, j] < A[i, j] < A[i+1, j]$ and $A[i, j-1] < A[i, j] < A[i, j+1]$ (assuming the indices are inside the array).

Devise a two-dimensional divide and conquer algorithm which takes a line-sorted array A and number k to search for, and returns either indices i and j such that $A[i, j] = k$ or an indicator that k is not in the array. Describe your algorithm (in pseudo-code), argue that your algorithm is correct, give and analyze its (worst-case) running time through a recurrence. You may assume that n is a power of two.

The goal is for your algorithm to take asymptotically less time than the number of elements in the array, but it will probably not be as efficient as simply performing binary search on each row.

- (a) Give the pseudocode for a divide-and-conquer algorithm to solve this problem, along with a short English description of the algorithm.

Solution. Function takes in a $n \times n$ array and target value to return the indices for the target value. The function works by splitting the 2D array into quadrants and choosing the appropriate ones to further search. The recursion function returns 3 values: the first 2 are the indices and the last is a boolean value to indicate value found.

```

1: function SEARCH2D(A, key , n)
2:   return search2D-r(A, key, 0, n - 1, 0, n - 1)
3: end function
4: function SEARCH2D-R(A, key, y1, y2, x1, x2)
5:   x  $\leftarrow \text{floor}(\frac{x_1+x_2}{2})$ 
6:   y  $\leftarrow \text{floor}(\frac{y_1+y_2}{2})$ 
7:   if A[x][y] == key then
8:     return x,y,true
9:   else if x == x1 || y == y1 then
10:    return NULL, NULL, false
11:  else if A[x][y] < key then
12:    x1, y1, ok1  $\leftarrow$  search2D-r(A, key, y + 1, y2, x1, x)           ▷ Top Right Quadrant
13:    x2, y2, ok2  $\leftarrow$  search2D-r(A, key, y1, y, x + 1, x2)         ▷ Bottom Left Quadrant
14:    x3, y3, ok3  $\leftarrow$  search2D-r(A, key, y + 1, y2, x + 1, x2)   ▷ Bottom Right Quadrant
15:    if ok1 then
16:      return x1, y1, ok1
17:    else if ok2 then
18:      return x2, y2, ok2
19:    else if ok3 then
20:      return x3, y3, ok3
21:    else
22:      return NULL, NULL, false
23:    end if
24:  else if A[x][y] > key
25:    x1, y1, ok1  $\leftarrow$  search2D-r(A, key, y + 1, y2, x1, x)           ▷ Top Right Quadrant
26:    x2, y2, ok2  $\leftarrow$  search2D-r(A, key, y1, y, x + 1, x2)         ▷ Bottom Left Quadrant
27:    x3, y3, ok3  $\leftarrow$  search2D-r(A, key, y1, y, x1, x)             ▷ Top Left Quadrant
28:    if ok1 then
29:      return x1, y1, ok1
30:    else if ok2 then
31:      return x2, y2, ok2
32:    else if ok3 then
33:      return x3, y3, ok3
34:    else
35:      return NULL, NULL, false
36:    end if
37:  end if
38: end function

```

(b) Prove the correctness of this algorithm.

Solution.

Base case:

Assume $n = 1$:

$\hookrightarrow$ tests true on line 9 and if tests true on line 7 returns (x, y, true) else returns $(\text{NULL}, \text{NULL}, \text{false})$ This is a 1×1 array in which there is only 1 element.

IH:

Given array A of $n \times n$ size and target value as val , the middle element is picked and assigned to var mid at $A[\frac{n}{2}][\frac{n}{2}]$. The cases we need to consider are as follows:

i. Case 1: $mid = val$

$\hookrightarrow$ The middle value of the 2D array is the target value. We just return the location of the element.

ii. Case 2: $mid < val$

$\hookrightarrow$ The middle value of the 2D array is less than the target value. We just return the location of the element, after searching through quadrants we know are greater than mid

iii. Case 3: $mid > val$

$\hookrightarrow$ The middle value of the 2D array is greater than the target value. We just return the location of the element, after searching through quadrants we know are less than mid

These are the three cases that the algorithm can follow. Now to define the actions for each case. Case 1 is our base case in which we found the target value so we return the position and a boolean value true . Case 2 and 3 are similar and both exclude a quadrant of the 2D array. We know that the 2D array is sorted in increasing order for each row AND column. This means that the value at $A[0][0]$ is the smallest and the value at $A[n-1][n-1]$ is the largest. We can split the 2D array into 4 equally sized quadrants (recursing continually makes the quadrants into more quadrants). From the properties of the 2D array we can say that the values in TopLeft quadrant are all less than mid and BottomRight greater than mid . The remaining two quadrants have values that are greater or lesser than mid . For this reason we cannot exclude the TopRight and BottomLeft quadrants as they may have the target value we are looking for. TopRight quadrant has values greater than mid the more right you go and values lesser than mid as you go up. Similarly BottomLeft has values lesser than mid the more left you go and values greater than mid the more you go down. Therefore the 2 quadrants have a possibility of containing the target value if $mid \neq \text{target}$ regardless of Case 2 or Case 3. The excluded quadrant is the one that is strictly greater or lesser relative to case 2 or 3. By this logic, we are guaranteed to find the value as the quadrants searched contain values that lead us closer to the target value.

- (c) Derive a recurrence for the algorithm, and analyze its worst-case runtime. Assume your input is an n -element matrix (i.e. a $\sqrt{n} \times \sqrt{n}$ matrix).

Solution.

$$T(n) = \begin{cases} 1 & n = 1 \\ 3 \cdot T(\frac{n}{4}) + 1 & n \geq 2 \end{cases}$$

This recurrence stems from the 3 recursive calls on 3 selected quadrants based on value comparisons. Since the 2D array is line sorted and column sorted we can look at the middle value and compare that to the value we need to look at and navigate in that direction. If mid is greater than the key val then we recurse on TopRight, BottomRight, and BottomLeft. BottomRight needs no explanation as that quadrant has values greater than any other quadrant. The reason that we pick quadrant TR and BL is because there exists values that can be greater than the middle value. BL going to down index $[n-1]$ increases and TR going right to index $[n-1]$ also increases. We do similat in the case that mid is less than val but recurse for TopLeft, BL and TR.

To get the bound for this runtime we can use the master's theorem:

MT:

$$T(n) = 3 \cdot T(\frac{n}{4}) + 1$$

In the form $T(n) = a \cdot T(\frac{n}{b}) + f(n)$

$$a = 3$$

$$b = 4$$

$$f(n) = 1 = n^0$$

Now we compare $n^{\log_b a}$ and n^0

$$\log_b a = \log_4 3 \approx 0.793$$

$$n^{\log_b a} \approx n^{0.793} \text{ vs } n^0 \Rightarrow n^{0.793} > n^0$$

since

$$n^{\log_b a} > n^0$$

we can move forward with Case 1:

"Look" for ϵ :

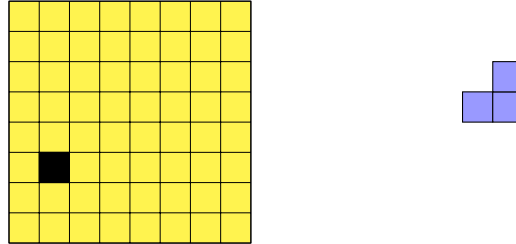
$$\epsilon = \log_4 3 = 0.793, \epsilon > 0$$

$$O(n^0) = O(n^{\log_4 3 - \epsilon}) = O(n^{0.793 - \epsilon})$$

By Case 1:

$$T(n) = \Theta(n^{\log_4 3}) = \Theta(n^{0.793})$$

2. *Tiling a Room.* You have a square room that is 2^k feet on a side (for some $k \geq 1$), so you can think of the room as a grid of $2^k \times 2^k$ cells. One of these cells is used for a floor vent. You are to find a way to fill in the $(2^k \times 2^k) - 1$ other cells perfectly with L-shaped tiles, like the one shown. The L-shaped tiles can be rotated in four different ways, but cannot be cut or extend outside the room. Give a divide and conquer algorithm that finds a tiling and argue that it is correct.



For full credit, your algorithms should take $O(n)$ time where $n = 2^{2k} = (2^k)^2$ is the number of cells in the room. The intent of the problem is to show that rooms of this kind can be tiled, do not worry about the book-keeping needed to record the particular solution found by your divide and conquer algorithm.

- (a) Give the pseudocode for a divide-and-conquer algorithm to solve this problem, along with a short English description of the algorithm.

Solution.

```

1: function TILI( $A, n, \text{tileX}, \text{tileY}$ )
2:   Tili-recurse( $A, \text{tileX}, \text{tileY}, n - 1, 0, n - 1$ )
3: end function
4: function TILI-RECURSE( $A, \text{tileX}, \text{tileY}, y_1, y_2, x_1, x_2$ )
5:   if  $\log_2((x_1 - x_2) + 1) == 1 \parallel \log_2((y_1 - y_2) + 1) == 1$  then
6:     place tile to cover the none - hole space(s)
7:     return
8:   else  $\log_2((x_1 - x_2) + 1) > 1 \parallel \log_2((y_1 - y_2) + 1) > 1$ 
9:      $x_{\text{mid}} \leftarrow \text{floor}(\frac{x_1 + x_2}{2})$ 
10:     $y_{\text{mid}} \leftarrow \text{floor}(\frac{y_1 + y_2}{2})$ 
11:    if  $\text{tileX} < x_{\text{mid}}$  then
12:      if  $\text{tileY} < y_{\text{mid}}$  then
13:        place 1 total tile on the innermost of the quadrants 1, 2, and 4
14:      else
15:        place 1 total tile on the innermost of the quadrants 1, 2, and 3
16:      end if
17:    else
18:      if  $\text{tileY} < y_{\text{mid}}$  then
19:        place 1 total tile on the innermost of the quadrants 1, 3, and 4
20:      else
21:        place 1 total tile on the innermost of the quadrants 2, 3, and 4
22:      end if
23:    end if
24:    Tili-recurse( $A, \text{tileX}, \text{tileY}, y_1, y_{\text{mid}}, x_1, x_{\text{mid}}$ )
25:    Tili-recurse( $A, \text{tileX}, \text{tileY}, y_1, y_2, x_1, x_2$ )
26:    Tili-recurse( $A, \text{tileX}, \text{tileY}, y_1, y_{\text{mid}}, x_{\text{mid}}, x_2$ )
27:    Tili-recurse( $A, \text{tileX}, \text{tileY}, y_{\text{mid}}, y_2, x_1, x_{\text{mid}}$ )
28:    Place tile to cover hole(s) created by lines 19-31
29:  end if
30: end function

```

(b) Prove the correctness of this algorithm.

Solution.

Base case:

Assume $k = 1$:

$\hookrightarrow$ tests true on line 5. This is a 2×2 array in which there is only 4 elements, of which one will be a hole. So Placing a tile to cover the rest of the spaces in the 2×2 sub array and returning is the only thing to do.

IH:

If we are to assume that the algorithm can successfully tile a room that is less than $2^k \times 2^k$ AND contains at least 1 vent for A is a $2^k \times 2^k$ array for $k \leq 2$, then we split the room into 4 quadrants and find the quadrant with the location of the initial vent. If we were to place a single tile such that all the quadrants virtually have vents (or cells that a tile cannot be placed on), we can apply the IH since now each quadrant (which can be considered as its own sub room for the algorithm) has a vent. By continuously dividing into quadrants and placing a single tile such that all the quadrants have a non-empty cell we can fully tile the room recursively.

(c) Derive a recurrence for the algorithm, and analyze its worst-case runtime.

Solution.

$$T(n) = \begin{cases} 1 & n \leq 4 \\ 4 \cdot T(\frac{n}{4}) + 1 & n > 4 \end{cases}$$

This recurrence stems from the 4 recursive calls on 4 quadrants. Since the 2D array is always a multiple of 4 ($2^n \times 2^n$) we can look at the middle value and compare that to the vent position and place a tile on the outside of that quadrant in the middle of the 2D sub-array. The numbering of the quadrants is as follows: TopRight-1, TopLeft-2, BottomLeft-3, BottomRight-4

To get the bound for this runtime we can use the master's theorem:

MT:

$$T(n) = 4 \cdot T(\frac{n}{4}) + 1$$

In the form $T(n) = a \cdot T(\frac{n}{b}) + f(n)$

$$a = 4$$

$$b = 4$$

$$f(n) = 1 = n^0$$

Now we compare $n^{\log_b a}$ and n^0

$$\log_b a = \log_4 4 = 1$$

$$n^{\log_b a} \approx n^1 \text{ vs } n^0 \Rightarrow n^1 > n^0$$

since

$$n^{\log_b a} > n^0$$

we can move forward with Case 1:

"Look" for ϵ :

$$\epsilon = \log_4 4 = 1, \epsilon > 0$$

$$O(n^0) = O(n^{\log_4 4 - \epsilon}) = O(n^{1 - \epsilon})$$

By Case 1:

$$T(n) = \Theta(n^{\log_4 4}) = \Theta(n^1) = \Theta(n)$$